



LINGUAGEM DART — PRIMEIRO CONTATO

Lista de Exercícios de Fixação

Programação para Dispositivos Móveis — 6º Período
Bacharelado em Sistemas de Informação
Instituto Federal Goiano — Campus Ceres

Dart 3.12 · todos os exercícios rodam em dartpad.dev

Prof. Dr. Paulo César Ferreira Melo



Orientações

Esta lista acompanha, na mesma ordem, os slides de sintaxe da linguagem Dart. São exercícios curtos, de fixação: cada um exercita um recurso específico e leva poucos minutos. Faça na sequência — os blocos finais pressupõem os anteriores.

Todos rodam em dartpad.dev, sem instalação. Escreva o código, execute, veja o resultado e só então passe adiante. Ler sobre sintaxe sem digitar não fixa nada.

Bloco	Tema	Exercícios
A	Variáveis, tipos e saída	A1 a A5
B	Segurança contra nulo	B1 a B5
C	Texto	C1 a C4
D	Decisão e repetição	D1 a D6
E	Coleções	E1 a E7
F	Funções	F1 a F5
G	Classes e objetos	G1 a G7
H	Enums e exceções	H1 a H4
I	Código assíncrono	I1 a I3
J	Integradores	J1 a J3

Como entregar

- Um arquivo por bloco, na pasta `exercicios/dart` do seu repositório: `bloco_a.dart`, `bloco_b.dart` e assim por diante.
- Separe os exercícios com um comentário identificando o número, e mantenha cada um em uma função própria chamada pelo main.
- Onde o enunciado pedir explicação, escreva em comentário no próprio arquivo.
- A numeração reinicia a cada bloco: identifique os exercícios pela letra do bloco mais o número, como A3 ou G7.

Contexto dos enunciados

Quase todos os exercícios usam dados de propriedades rurais da região de Ceres e do Vale de São Patrício. Não é enfeite: os mesmos dados reaparecem no aplicativo que construiremos no semestre, o Caderno de Campo do Vale.

Referência permanente para consulta: dart.dev/language. Consultar documentação durante os exercícios é esperado, não é trapaça.

Bloco A — Variáveis, tipos e saída

- Declare, com tipos explícitos, o nome de um produtor, a cidade, a área da propriedade em hectares e o ano da safra. Imprima uma linha de identificação com os quatro valores.



2. Refaça o exercício anterior usando `var` em todas as declarações. Escreva em comentário qual tipo foi inferido em cada caso.
3. Declare uma constante para o alqueire goiano (4,84 ha) e outra variável com `final` recebendo `DateTime.now()`. Tente trocar o `final` por `const`, leia o erro e explique a diferença entre os dois.
4. Converta uma área em hectares para alqueires goianos e imprima o resultado com duas casas decimais.
5. Declare três cotações de saca. Imprima a maior, a menor e a média, todas com duas casas. Dica: `a > b ? a : b` resolve a comparação de dois valores.

Bloco B — Segurança contra nulo

1. Declare `String? telefone` sem valor inicial e imprima-a usando `??` para exibir "não informado" quando estiver vazia.
2. Tente imprimir `telefone.length` diretamente. Copie a mensagem de erro como comentário e depois corrija usando `?.`
3. Escreva uma função que receba `double? chuvaMm` e devolva um texto: "sem registro" quando for nula, "seca" abaixo de 20 mm e "normal" acima disso.
4. Escreva uma função com retorno `double?` que calcule sacas por hectare e devolva nulo quando a área for zero ou negativa. Trate o resultado no `main` com uma verificação explícita, sem usar o operador `!`.
5. Force uma quebra: use o operador `!` sobre um valor que você sabe ser nulo. Copie a mensagem de execução e escreva, em comentário, por que esse operador deve ser evitado.

Bloco C — Texto

1. Usando interpolação, monte e imprima a frase "O Talhão 3 tem 42,0 ha plantados com milho" a partir de três variáveis.
2. Imprima um valor monetário no formato brasileiro, com o cifrão escapado corretamente e vírgula como separador decimal. Dica: `toStringAsFixed` seguido de `replaceAll`.
3. Crie um texto de várias linhas com o endereço completo de uma cooperativa e imprima-o.
4. A partir da string `' soja , milho , sorgo '`, produza uma lista com as três culturas sem espaços em excesso e em letras maiúsculas.

Bloco D — Decisão e repetição

1. Classifique uma produtividade em três faixas: abaixo de 50 sc/ha, entre 50 e 70, e acima de 70. Use `if/else`.
2. Refaça a classificação anterior usando `switch` como expressão (a forma com `=>` que devolve um valor).
3. Imprima uma tabela de conversão de hectares para alqueires goianos, de 10 em 10, de 10 até 100 hectares.
4. Simule a colheita: partindo de 5.000 sacas em estoque, retire 350 por dia e imprima quantos dias levam até o estoque acabar. Use `while`.
5. Percorra uma lista de meses da safra com `for-in` e imprima cada um numerado.



6. Percorra uma lista de áreas e some apenas as maiores que zero, usando `continue` para pular as demais. Interrompa o laço com `break` se a soma passar de 200 ha.

Bloco E — Coleções

1. Crie uma lista de culturas. Acrescente uma, remova outra e imprima o tamanho, o primeiro e o último elemento.
2. Crie uma lista com culturas repetidas e converta-a em um conjunto para eliminar as repetições. Imprima os dois e compare.
3. Crie um mapa de cultura para cotação da saca. Acrescente uma nova entrada, consulte uma existente e consulte uma que não existe — observe o que o Dart devolve neste último caso.
4. Percorra o mapa do exercício anterior com `entries` e imprima uma linha por cultura, no formato "soja: R\$ 128,40".
5. A partir de uma lista de áreas, use `where` para separar as maiores que 30 ha, `map` para convertê-las em alqueires e `fold` para somar o total. Faça em uma única expressão encadeada.
6. Sobre a mesma lista, responda com `any`, `every` e `firstWhere`: existe algum talhão acima de 40 ha? todos têm mais de 10 ha? qual é o primeiro abaixo de 20 ha?
7. Ordene a lista de áreas do maior para o menor com `sort` e `compareTo`. Depois inverta a ordem alterando apenas a ordem dos operandos.

Bloco F — Funções

1. Escreva uma função que receba a área em hectares e devolva o valor em alqueires goianos. Escreva também a versão com sintaxe de seta.
2. Escreva uma função de formatação que receba um número e, opcionalmente, o número de casas decimais, com 1 como padrão. Chame-a das duas formas: com e sem o segundo argumento.
3. Escreva uma função com parâmetros nomeados: nome e área são obrigatórios (`required`), cultura tem valor padrão. Chame-a invertendo a ordem dos argumentos e confirme que funciona.
4. Escreva uma função que receba uma lista de áreas e uma função de transformação, aplique a transformação a cada elemento e imprima o resultado. Chame-a duas vezes, com transformações diferentes.
5. Escreva uma função que calcule a receita bruta a partir de área, produtividade e preço da saca, com os três parâmetros nomeados e obrigatórios.

Bloco G — Classes e objetos

1. Crie a classe `Produtor` com nome, cidade e telefone (que pode não existir). Use campos `final` e construtor com parâmetros nomeados obrigatórios.
2. Acrescente um construtor nomeado `Produtor.semTelefone` que dispense o telefone.
3. Crie a classe `Talhao` com nome, área e cultura. Acrescente um campo calculado que devolva a área em alqueires e outro que informe se o talhão tem menos de 20 ha.
4. Sobrescreva `toString` em `Talhao` e imprima um objeto diretamente com `print`.



5. Acrescente à classe uma constante `static` com o valor do alqueire goiano e use-a no campo calculado. Depois acesse essa constante a partir do `main`, sem criar nenhum objeto.
6. Crie uma lista de objetos `Talhao` e repita sobre ela as agregações do Bloco E: área total, filtro por cultura e o maior talhão. Compare com a versão que usava mapas.
7. Crie uma classe abstrata `Cultura` com nome e ciclo em dias, e duas subclasses concretas. Guarde as duas em uma lista do tipo da classe abstrata e percorra imprimindo o resumo de cada uma.

Bloco H — Enums e exceções

1. Crie um enum `Atividade` com plantio, adubação, pulverização e colheita. Use-o em um `switch` que devolva uma descrição de cada atividade.
2. Transforme o enum anterior em um enum com campos, guardando também um rótulo em português e se a atividade exige registro de defensivo.
3. Escreva uma função que valide a área informada e lance `ArgumentError` quando ela for menor ou igual a zero. Chame-a dentro de `try/catch` e trate o erro.
4. Acrescente um bloco `finally` ao exercício anterior e confirme, por experimento, que ele executa tanto no caso de sucesso quanto no de erro.

Bloco I — Código assíncrono

1. Escreva uma função que simule a leitura de um sensor de umidade do solo: aguarde dois segundos com `Future.delayed` e devolva um valor. Chame-a com `await` e imprima mensagens antes e depois.
2. Chame a mesma função sem `await` e imprima o que é devolvido. Explique em comentário o que apareceu no console.
3. Faça três leituras em sequência e depois três simultâneas com `Future.wait`. Meça os dois tempos com `Stopwatch` e explique a diferença.

Bloco J — Integradores

Os três últimos exercícios combinam recursos de vários blocos. São o fechamento da lista e a preparação direta para o que faremos em Flutter.

1. Relatório da propriedade: a partir de uma lista de objetos `Talhao`, imprima um relatório com área total, área por cultura, percentual de cada cultura e o talhão de maior área. Toda a formatação deve sair no padrão brasileiro.
2. Registro de atividades: crie uma classe `Registro` com talhão, atividade (enum), data e observação opcional. Monte uma lista de registros, valide a data com exceção e imprima o histórico agrupado por talhão.
3. Receita estimada: junte a lista de talhões a uma consulta simulada de cotação, feita de forma assíncrona. Calcule a receita bruta estimada de cada talhão e o total da propriedade, tratando o caso em que a consulta falha.

O que estes três exercícios antecipam



O relatório é a lógica que estará por trás de uma tela de resumo. O registro de atividades é o núcleo do Caderno de Campo do Vale. E a receita estimada é a primeira vez em que dados locais e dados de rede aparecem juntos — o problema central de qualquer aplicativo que funcione no meio da lavoura.

Prof. Dr. Paulo César Ferreira Melo — paulo.melo1@ifgoiano.edu.br